

Présentation de Big Data

Sujet: Etude et Installation d'un Spark en Big Data

Présenté par:

M. Ulrich Noéjules OGO NDAYEGUIA

I- INTRODUCTION

1. Introduction à Apache Spark

- ▶ **Apache Spark** est un moteur de traitement distribué rapide et généraliste pour le **Big Data**. Il a été développé pour surpasser **Hadoop MapReduce** en termes de rapidité et de simplicité, en particulier pour les opérations itératives comme les algorithmes de machine Learning.
- ▶ Spark permet de traiter de grandes quantités de données dans un cluster distribué, en offrant des API simples dans plusieurs langages : **Scala, Python, Java**, et **R**. Il est également compatible avec **Hadoop** et peut utiliser **HDFS, YARN**, et d'autres systèmes de stockage distribués.

II- DEVELOPPEMENT

2. Architecture d'Apache Spark

- ▶ L'architecture de Spark est basée sur un modèle de traitement **maître-esclave**. Un programme Spark est composé de **Driver**, de **Cluster Manager**, et de **Workers** :
- **Driver** : Le processus principal qui orchestre l'exécution des tâches. Il convertit le programme Spark en un **DAG (Directed Acyclic Graph)** d'étapes à exécuter.
- **Cluster Manager** : Responsable de la gestion des ressources. Il peut s'agir de **Standalone Spark**, **YARN**, ou **Mesos**.
- **Workers** : Exécutent les tâches envoyées par le Driver. Chaque Worker contient plusieurs **Executors** qui exécutent les instructions de calcul.

3. Composants clés de Spark

- ▶ Spark se compose de plusieurs modules spécialisés pour différents cas d'utilisation.

a) Spark Core

- ▶ C'est le cœur de Spark. Il fournit les API fondamentales pour le traitement des données en mémoire distribuée et inclut des fonctionnalités pour la gestion des tâches, l'ordonnancement et la récupération.

b) Spark SQL

- ▶ C'est un module pour l'analyse structurée des données. Il permet d'interagir avec Spark à l'aide de SQL et prend en charge les **DataFrames** et **Datasets**, qui offrent des abstractions plus performantes que les RDD (Resilient Distributed Datasets).

c) Spark Streaming

- ▶ Ce module permet le traitement de flux de données en temps réel. Il est utilisé pour traiter des données en temps réel, comme les logs ou les événements d'applications, en micro-lots.

d) MLlib

MLlib est la bibliothèque d'apprentissage automatique distribuée de Spark. Elle inclut des algorithmes pour la régression, la classification, le clustering et la recommandation.

e) GraphX

- ▶ C'est un module pour les calculs distribués sur les graphes. Il permet de représenter des graphes de données et d'exécuter des algorithmes sur ces graphes, comme **PageRank** ou des recherches sur des chemins.

4. Installation d'Apache Spark

a) Prérequis

- **Java JDK 8+**
- **Scala** (optionnel, mais recommandé)
- **Python** (si vous voulez utiliser PySpark)
- **Hadoop** (optionnel, si vous voulez intégrer Spark à un cluster Hadoop)

b) Installation

- ▶ Voir la section complète pour l'installation de Spark sous Ubuntu dans la partie pratique à partir de la page 11 .

5. Les concepts fondamentaux

a) RDD (Resilient Distributed Dataset)

- ▶ Les **RDD** sont la structure de base de Spark pour représenter les données. Ils sont distribués à travers plusieurs nœuds d'un cluster et sont **résilients** car ils permettent la reconstitution des données en cas de panne.

b) DataFrame

- ▶ Les **DataFrames** sont une abstraction au-dessus des RDD, représentant des données structurées (comme des tables de bases de données). Ils sont optimisés pour des traitements rapides en utilisant un optimiseur interne appelé **Catalyst**.

c) Dataset

- ▶ Les **Datasets** sont similaires aux DataFrames mais fournissent des vérifications de type à la compilation, ce qui permet d'obtenir un code plus sûr.

5. Les concepts fondamentaux

d) Transformation vs Action

- ▶ Transformation : Opérations qui créent de nouveaux RDD (ex : `map()`, `filter()`),
- ▶ mais qui ne sont pas exécutées immédiatement (car Spark utilise un modèle d'évaluation paresseuse).
- ▶ Action : Opérations qui déclenchent l'exécution des transformations (ex : `collect()`, `count()`).

6. Modes de déploiement

- ▶ Spark peut être déployé dans différents environnements :
- **Standalone** : Spark s'exécute en tant que gestionnaire de cluster autonome.
- **YARN** : Spark est déployé au-dessus de YARN (Hadoop).
- **Mesos** : Spark utilise Mesos comme gestionnaire de ressources.
- **Kubernetes** : Spark peut être déployé en conteneurs sur Kubernetes.

III- PRATIQUE

Installation complète de Spark sous Ubuntu

Vérification et fonctionnement du DNS de notre serveur Spark

```
root@spark: /etc/bind
root@spark:/etc/bind# nslookup www
Server:      192.168.229.131
Address:     192.168.229.131#53

www.examen.cf  canonical name = spark.examen.cf.
Name:   spark.examen.cf
Address: 192.168.229.131

root@spark:/etc/bind# ping google.fr
PING google.fr (216.58.215.131) 56(84) bytes of data.
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=1 ttl=128 time=64.9 ms
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=2 ttl=128 time=165 ms
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=3 ttl=128 time=229 ms
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=4 ttl=128 time=145 ms
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=5 ttl=128 time=76.8 ms
64 bytes from mad41s04-in-f3.1e100.net (216.58.215.131): icmp_seq=6 ttl=128 time=91.8 ms
^C
--- google.fr ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5006ms
rtt min/avg/max/mdev = 64.907/128.644/228.974/57.329 ms
root@spark:/etc/bind# ping google.com
PING google.com (142.250.201.78) 56(84) bytes of data.
64 bytes from mad07s25-in-f14.1e100.net (142.250.201.78): icmp_seq=1 ttl=128 time=60.8 ms
64 bytes from mad07s25-in-f14.1e100.net (142.250.201.78): icmp_seq=2 ttl=128 time=60.6 ms
64 bytes from mad07s25-in-f14.1e100.net (142.250.201.78): icmp_seq=3 ttl=128 time=104 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 60.620/75.120/103.903/20.352 ms
root@spark:/etc/bind#
```

Installation de openjdk-8-jdk

```

root@spark: ~
root@spark:~#
root@spark:~# apt install openjdk-8-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Les paquets supplémentaires suivants seront installés :
adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-core ca-certificates-java dconf-gsettings-backend
dconf-service fontconfig fonts-dejavu-extra gsettings-desktop-schemas gtk-update-icon-cache hicolor-icon-theme
humanity-icon-theme java-common libasound2 libasound2-data libasyncns0 libatk-bridge2.0-0 libatk-wrapper-java
libatk-wrapper-java-jni libatk1.0-0 libatk1.0-data libatspi2.0-0 libavahi-client3 libavahi-common-data
libavahi-common3 libcairo-gobject2 libcairo2 libcups2 libdatrie1 libdconf1 libdrm-amdgpu1 libdrm-intel1
libdrm-nouveau2 libdrm-radeon1 libflac8 libfontenc1 libgail-common libgail18 libgdk-pixbuf-2.0-0
libgdk-pixbuf2.0-bin libgdk-pixbuf2.0-common libgif7 libgl1 libgl1-amd-gpu libgl1-mesa-dri libgl1-mesa-glx
libglapi-mesa libglvnd0 libglx-mesa0 libglx0 libgraphite2-3 libgtk2.0-0 libgtk2.0-bin libgtk2.0-common libharfbuzz0b
libice-dev libice6 liblcms2-2 libllvm15 libogg0 libopus0 libpango-1.0-0 libpangocairo-1.0-0 libpangoft2-1.0-0
libpciaccess0 libpcsclite1 libpixmap-1-0 libpthread-stubs0-dev libpulse0 librsvg2-2 librsvg2-common
libsensors-config libsensors5 libsm-dev libsm6 libsndfile1 libthai-data libthai0 libvorbis0a libvorbisenc2 libx11-6
libx11-dev libx11-xcb1 libxau-dev libxaw7 libxcb-dri2-0 libxcb-dri3-0 libxcb-glx0 libxcb-present0 libxcb-randr0
libxcb-render0 libxcb-shape0 libxcb-shm0 libxcb-sync1 libxcb-xfixes0 libxcb1-dev libxcomposite1 libxcursor1
libxdamage1 libxdmcp-dev libxfixes3 libxft2 libxi6 libxinerama1 libxkbfile1 libxmu6 libxrandr2 libxrender1
libxshmfence1 libxt-dev libxt6 libxtst6 libxv1 libxxf86dga1 libxxf86vm1 openjdk-8-jdk-headless openjdk-8-jre
openjdk-8-jre-headless session-migration ubuntu-mono x11-common x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev
Paquets suggérés :
default-jre libasound2-plugins alsa-utils cups-common gvfs libice-doc liblcms2-utils opus-tools pscd pulseaudio
librsvg2-bin lm-sensors libsm-doc libx11-doc libxcb-doc libxt-doc openjdk-8-demo openjdk-8-source visualvm
libnss-mdns fonts-nanum fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei fonts-wqy-zenhei fonts-indic
mesa-utils
Les NOUVEAUX paquets suivants seront installés :
adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-core ca-certificates-java dconf-gsettings-backend
dconf-service fontconfig fonts-dejavu-extra gsettings-desktop-schemas gtk-update-icon-cache hicolor-icon-theme

```

Vérification de la version openjdk installée

```
root@spark: ~  
root@spark:~# java -version  
openjdk version "1.8.0_422"  
OpenJDK Runtime Environment (build 1.8.0_422-8u422-b05-1~22.04-b05)  
OpenJDK 64-Bit Server VM (build 25.422-b05, mixed mode)  
root@spark:~#  
root@spark:~#  
root@spark:~#  
root@spark:~#  
root@spark:~#
```

Installation de scala

```
root@spark: ~  
See https://ubuntu.com/esm or run: sudo pro status  
New release '24.04.1 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Wed Oct  9 19:29:10 2024 from 192.168.229.1  
root@spark:~# apt install scala -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Les paquets supplémentaires suivants seront installés :  
  libhawtjni-runtime-java libjansi-java libjansi-native-java libjline2-java scala-library scala-parser-combinators  
  scala-xml  
Paquets suggérés :  
  scala-doc  
Les NOUVEAUX paquets suivants seront installés :  
  libhawtjni-runtime-java libjansi-java libjansi-native-java libjline2-java scala scala-library  
  scala-parser-combinators scala-xml  
0 mis à jour, 8 nouvellement installés, 0 à enlever et 178 non mis à jour.  
Il est nécessaire de prendre 25,1 Mo dans les archives.  
Après cette opération, 28,6 Mo d'espace disque supplémentaires seront utilisés.  
Réception de :1 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 libhawtjni-runtime-java all 1.17-1 [28,8 kB]  
Réception de :2 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 libjansi-native-java all 1.8-1 [23,8 kB]  
Réception de :3 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 libjansi-java all 1.18-1 [56,8 kB]  
Réception de :4 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 libjline2-java all 2.14.6-4 [150 kB]  
Réception de :5 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-library all 2.11.12-5 [90586 kB]  
Réception de :6 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-parser-combinators all 1.0.3-3.1 [365 kB]  
Réception de :7 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-xml all 1.0.3-3.1 [615 kB]  
Réception de :8 http://sn.archive.ubuntu.com/ubuntu jammy/universe amd64 scala all 2.11.12-5 [14,2 MB]
```


Téléchargement de Spark en ligne

```
root@spark: ~  
root@spark:~# wget https://dlcdn.apache.org/spark/spark-3.4.3/spark-3.4.3-bin-hadoop3.tgz  
--2024-10-09 19:37:42-- https://dlcdn.apache.org/spark/spark-3.4.3/spark-3.4.3-bin-hadoop3.tgz  
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644  
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 388930980 (371M) [application/x-gzip]  
Saving to: 'spark-3.4.3-bin-hadoop3.tgz'  
  
spark-3.4.3-bin-hadoop3.tgz 100%[=====>] 370,91M 2,31MB/s in 2m 22s  
2024-10-09 19:40:05 (2,61 MB/s) - 'spark-3.4.3-bin-hadoop3.tgz' saved [388930980/388930980]  
  
root@spark:~# ls  
snap spark-3.4.3-bin-hadoop3.tgz  
root@spark:~#  
root@spark:~#  
root@spark:~#  
root@spark:~#  
root@spark:~#
```

Décompression de Spark .tgz

```
root@spark: ~  
root@spark:~# tar -xvf spark-3.4.3-bin-hadoop3.tgz  
spark-3.4.3-bin-hadoop3/  
spark-3.4.3-bin-hadoop3/R/  
spark-3.4.3-bin-hadoop3/R/lib/  
spark-3.4.3-bin-hadoop3/R/lib/sparkr.zip  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/html/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/html/R.css  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/html/00Index.html  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/worker/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/worker/worker.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/worker/daemon.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/tests/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/tests/testthat/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/tests/testthat/test_basic.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/profile/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/profile/shell.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/profile/general.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/doc/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/doc/index.html  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/doc/sparkr-vignettes.html  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/doc/sparkr-vignettes.Rmd  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/doc/sparkr-vignettes.R  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/R/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/R/SparkR  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/R/SparkR.rdx  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/R/SparkR.rdb  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/Meta/  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/Meta/nsInfo.rds  
spark-3.4.3-bin-hadoop3/R/lib/SparkR/Meta/vignette.rds
```

Export du chemin de Spark dans ~/.bashrc

```
root@spark: ~  
alias grep='grep --color=auto'  
alias fgrep='fgrep --color=auto'  
alias egrep='egrep --color=auto'  
fi  
# some more ls aliases  
alias ll='ls -alF'  
alias la='ls -A'  
alias l='ls -CF'  
# Alias definitions.  
# You may want to put all your additions into a separate file like  
# ~/.bash_aliases, instead of adding them here directly.  
# See /usr/share/doc/bash-doc/examples in the bash-doc package.  
if [ -f ~/.bash_aliases ]; then  
    . ~/.bash_aliases  
fi  
# enable programmable completion features (you don't need to enable  
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile  
# sources /etc/bash.bashrc).  
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then  
#    . /etc/bash_completion  
#fi  
export SPARK_HOME=/opt/spark  
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin  
~/.bashrc" 103L, 3188B
```


IV- CONCLUSION

- ▶ **Apache Spark** est un outil puissant et polyvalent pour traiter des données massives en distribuant les tâches sur un cluster de machines. Grâce à son architecture basée sur la mémoire, il surpasse largement Hadoop pour certaines tâches et est largement utilisé pour l'analyse des données, le machine Learning et le traitement en flux.